Dokumentace

KIV/UPS - Semestrální práce

Gaétan Thomassin

9. ledna 2026

Obsah

1 Základní popis hry **3**

2 Popis komunikačního protokolu **3**
2.1 Formát zpráv 3
2.2 Datové typy a struktury 3
2.3 Význam přenášených dat a kódů 4
2.4 Stavový diagram 6
2.5 Validace a chybové stavy 6

3 Popis implementace **6**
3.1 Server (Jazyk C) 6
3.2 Klient (Java) 8

4 Požadavky na překlad a běh **8**

5 Postup překladu a spuštění **9**
5.1 Server 9
5.2 Klient 9

6 Závěr a zhodnocení **10**

1 Základní popis hry

Projekt implementuje klasickou deskovou hru **Šachy** pro dva hráče v architektuře klient-server. Cílem bylo vytvořit stabilní síťovou aplikaci, která respektuje pravidla hry a zároveň je odolná vůči běžným síťovým problémům.

Hra plně respektuje pravidla FIDE s implementací následujících klíčových mechanik:

- Pohyb všech standardních figur (pěšec, věž, jezdec, střelec, dáma, král).
- Ověřování legálnosti tahů (nelze táhnout do šachu, blokování cesty, specifika pohybu pěšců).
- Speciální tahy: **Rošáda** (malá i velká), **Braní mimochodem** (En Passant), **Proměna pěšce** (Promotion).
- Detekce herních stavů: **Šach**, **Mat**, **Pat**.

Hra končí matem, patem, vzdáním se (rezignací), vypršením časového limitu na tah, nebo vzájemnou dohodou o remíze. Z pohledu síťové architektury se hráči připojují do centrální "Lobby", kde mohou zakládat nové místnosti nebo se připojovat do existujících. Důležitou součástí je mechanismus zotavení po výpadku (rekonekce), který umožňuje hráči návrat do rozehrané partie bez ztráty stavu hry.

2 Popis komunikačního protokolu

Komunikace probíhá pomocí vlastního textového protokolu nad transportní vrstvou **TCP**. Protokol je navržen jako řádkově orientovaný (ukončovací znak `\n`), což usnadňuje ladění a čitelnost.

2.1 Formát zpráv

Struktura přenášených dat je navržena s důrazem na jednoduchost a snadné parsování. Základní pravidla formátování zpráv jsou následující:

- **Kódování:** Zprávy jsou přenášeny v kódování UTF-8 (kompatibilní s ASCII).
- **Příkaz:** Každá zpráva začíná klíčovým slovem příkazu:
  `PRIKAZ <parametr1> <parametr2> ...`
- **Oddělovač:** Jednotlivé parametry jsou odděleny mezerou.
- **Ukončení:** Každá zpráva je striktně ukončena znakem nového řádku (`\n`).

Tento formát umožňuje zpracování zpráv pomocí standardních funkcí pro práci s řetězci na obou stranách komunikace.

2.2 Datové typy a struktury

Pro zajištění konzistence dat mezi klientem a serverem jsou definovány specifické datové typy a formáty identifikátorů. Mezi klíčové přenášené struktury patří:

- **Souřadnice:** Používá se standardní algebraická notace (např. `e2e4` pro běžný tah, `a7a8q` pro proměnu pěšce na dámu).

- **SessionID:** Unikátní řetězec (UUID) generovaný klientem, který slouží k trvalé identifikaci relace a umožňuje rozpoznání hráče při opětovném připojení.
- **RoomID:** Celé číslo jednoznačně identifikující konkrétní herní místnost na serveru.
- **ACK kódy:** Dvouznakové číselné řetězce (např. `05`, `18`), které slouží k potvrzení přijetí a zpracování zprávy na aplikační vrstvě.

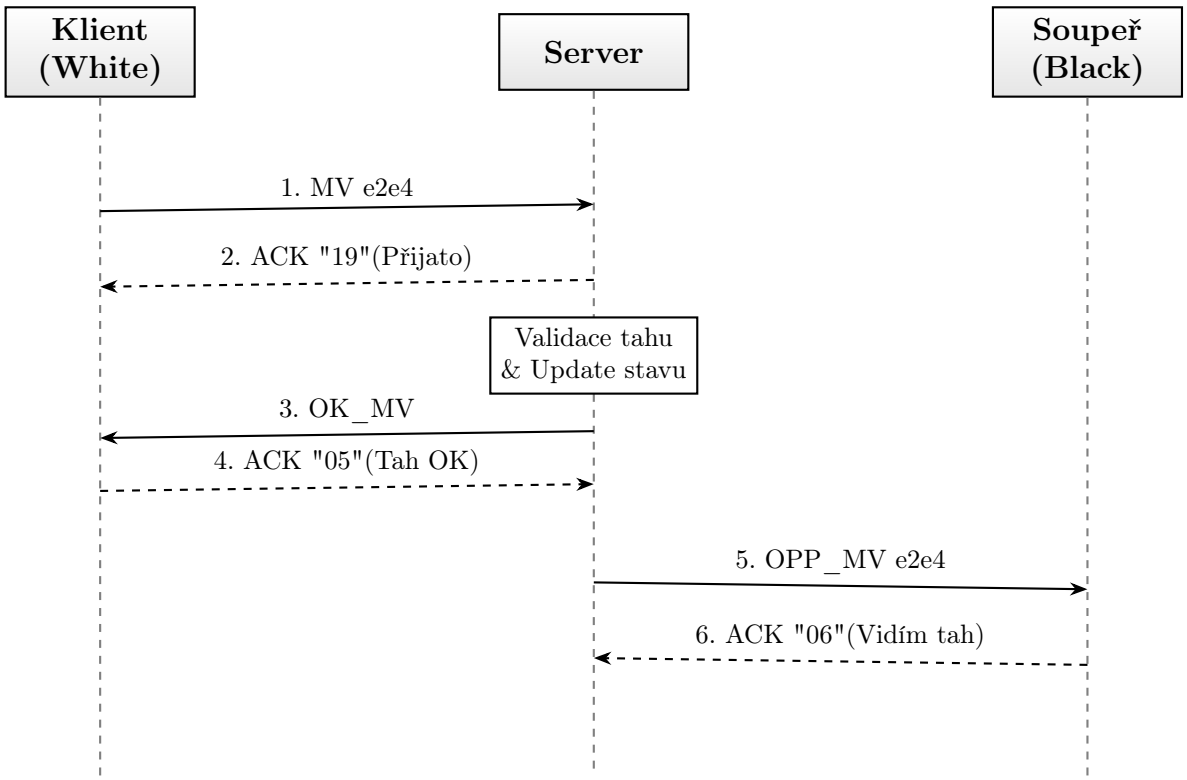
2.3 Význam přenášených dat a kódů

Komunikace je rozdělena na příkazy iniciované klientem, odpovědi serveru a potvrzovací mechanismus.

Klient → Server (Příkazy)

Klient řídí svůj stav a interakci s hrou pomocí sady definovaných příkazů. Seznam dostupných příkazů zahrnuje:

`HELLO <přezdívka> <ID>`	Úvodní handshake a identifikace hráče.
`LOBBY`	Požadavek na opuštění hry/čekání a návrat do lobby.
`LIST`	Požadavek na zaslání aktuálního seznamu místností.
`NEW`	Příkaz pro vytvoření nové herní místnosti.
`JOIN <ID místnosti>`	Pokus o připojení do existující místnosti.
`MV <od><kam>[p]`	Odeslání herního tahu (např. `MV e2e4`).
`RES`	Oznámení o rezignaci (vzdání se).
`DRW_OFF/ACC/DEC`	Sada příkazů pro nabídku remízy, její přijetí či odmítnutí.
`PING`	Keep-alive zpráva pro udržení spojení.
`EXT`	Příkaz pro úplné ukončení relace nebo odchod ze hry.



Obrázek 1: Sekvenční diagram komunikace při herním tahu (White táhne e2e4)

Server → Klient (Odpovědi a Notifikace)

Server reaguje na příkazy klienta nebo asynchronně notifikuje o změnách stavu. Hlavní zprávy serveru jsou:

- `WELCOME`: Potvrzení úspěšného navázání spojení.
- `ROOMLIST <id:jmeno ...>`: Odpověď obsahující seznam aktivních místností.
- `WAITING Room <id>`: Informace, že hráč úspěšně založil místnost a čeká na soupeře.
- `START <souper> <barva>`: Notifikace o začátku hry, přiřazení soupeře a barvy figur.
- `OK_MV`: Potvrzení, že odeslaný tah byl validní a proveden.
- `OPP_MV <tah>`: Notifikace, že soupeř provedl tah.
- `CHK`, `CHKM`, `SM`: Informace o změně herního stavu (šach, mat, pat).
- `TIME <sekundy>`: Aktualizace zbývajícího času na tah.
- `ERR <zpráva>`: Chybové hlášení (např. neplatný tah, plný server).
- `FULL`: Indikace, že server dosáhl maximální kapacity (následuje odpojení).
- `WAIT_CONN`: Informace o tom, že se soupeř odpojil a běží ochranná lhůta pro jeho návrat.
- `RESUME ...`: Potvrzení, že hra byla úspěšně obnovena po výpadku spojení.

Acknowledgement (ACK) mechanismus

Pro zvýšení spolehlivosti protokolu nad rámec TCP je zavedena aplikační vrstva potvrzování. Každá validní zpráva je potvrzena druhou stranou pomocí ACK kódu:

- `18`: HELLO_ACK (Handshake OK)
- `05`: ACCEPT_MOVE_ACK (Tah přijat a zpracován)
- `02`: WAIT_ACK (Přechod do stavu čekání v lobby)
- `99`: GENERIC_ACK (Obecné potvrzení)

Tento mechanismus zajišťuje, že odesílatel má jistotu o sémantickém zpracování příkazu, nikoliv pouze o jeho doručení na síťovou vrstvu.

2.4 Stavový diagram

Životní cyklus klienta je řízen stavovým automatem, který zajišťuje validní posloupnost operací.

1. **STATE_HANDSHAKE**: Výchozí stav. Klient se připojí a posílá `HELLO`. Server ověří kapacitu. Pokud existuje stará relace, provede se reconnect, jinak se vytvoří nová.
2. **STATE_LOBBY**: Hráč je autentizován. Vidí seznam her (`LIST`), může založit novou (`NEW` → přechod do WAITING), nebo se připojit (`JOIN` → přechod do GAME).
3. **STATE_WAITING**: Hráč založil místnost a čeká na soupeře. Tento stav lze zrušit (`EXT` → zpět do LOBBY).
4. **STATE_GAME**: Probíhá aktivní hra. Dochází k výměně zpráv `MV`. Hra končí stavem mat, rezignace nebo timeoutem, poté následuje návrat do LOBBY.
5. **STATE_DISCONNECTED**: Konečný stav, kdy je socket uzavřen a zdroje uvolněny.

2.5 Validace a chybové stavy

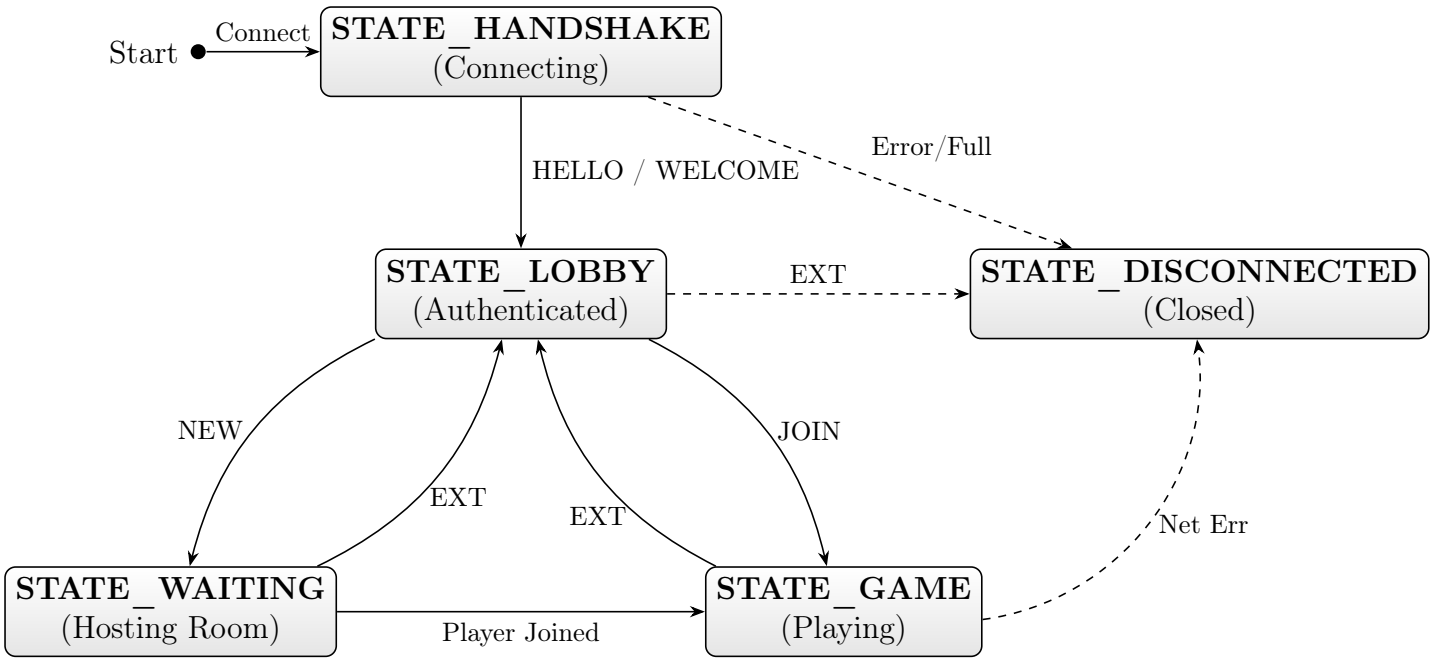
Robustnost systému je zajištěna dvoufázovou kontrolou dat a definovaným chováním při porušení protokolu.

- **Validace:** Server provádí striktní kontrolu vstupů. Kontroluje se délka zpráv (ochrana proti buffer overflow), formát příkazů (pomocí `strncmp` a `sscanf`) a herní logika (validátor pohybů v modulu `game.c`).
- **Chybové stavy:** Pokud klient odešle nevalidní příkaz, server odpoví zprávou `ERR <popis>`. Server si udržuje interní počítadlo chyb (`error_count`) pro každého klienta. **Při překročení limitu (např. 3 nevalidní zprávy) je klient považován za chybového nebo škodlivého, je okamžitě odpojen a jeho soupeř vyhrává kontumačně (`OPP_KICK`).**

3 Popis implementace

3.1 Server (Jazyk C)

Serverová část je implementována v jazyce C s důrazem na výkon a využití nízkoúrovňových systémových prostředků (POSIX vlákna, BSD sockety).



Obrázek 2: UML Stavový diagram klienta

Dekompozice do modulů

Zdrojový kód serveru je rozdělen do několika modulů podle funkční odpovědnosti, což usnadňuje údržbu a rozšiřitelnost:

- `main.c`: Vstupní bod aplikace. Inicializuje socket, naslouchá na portu (bind/listen), přijímá spojení (accept) a pro každého klienta vytváří nové vlákno.
- `client.c`: Obsluhuje životní cyklus konkrétního klienta (funkce `client_worker`). Implementuje stavový automat (FSM) a zajišťuje parsování i odesílání zpráv.
- `match.c`: Zodpovídá za správu herních místností. Udržuje globální seznam aktivních her a spouští vlákno `match_watchdog`, které hlídá časové limity a řeší odpojení hráčů.
- `game.c`: Obsahuje čistou logiku šachu. Definuje reprezentaci šachovnice `GameState`, validuje tahy a detekuje stavy jako šach či mat.
- `logging.c`: Zajišťuje thread-safe logování do souboru a na konzoli pomocí fronty zpráv a dedikovaného zapisovacího vlákna.

Metoda paralelizace

Pro zajištění plynulého chodu více her současně a rychlé odezvy byl zvolen model vícevláknového zpracování:

- **Per-Client Thread:** Každý připojený klient je obsluhován vlastním vláknem. To umožňuje blokující čtení ze socketu bez ovlivnění ostatních hráčů.
- **Watchdog Thread:** Každá běžící hra má přiřazeno pomocné vlákno pro správu časovačů a timeoutů.
- **Synchronizace:** Přístup ke sdíleným datům (seznam hráčů, stav hry v `match` strukturách) je chráněn pomocí mutexů (`pthread_mutex_t`), aby nedocházelo k souběhu (race conditions).

3.2 Klient (Java)

Klientská část je realizována jako desktopová aplikace v jazyce Java (Swing), kompatibilní s Java 8.

Rozvrstvení aplikace

Architektura klienta odděluje logiku hry, síťovou komunikaci a grafické rozhraní:

- **Networking (`NetworkClient.java`):** Vrstva nízkoúrovňové komunikace. Spravuje vlákna pro čtení (`readLoop`), zápis (`writeLoop`) a heartbeat. Implementuje ACK handshake vrstvu.
- **Controller (`ChessClient.java`):** Hlavní řídící třída. Propojuje síťovou vrstvu s GUI, řídí přepínání obrazovek (CardLayout) a interpretuje zprávy ze serveru.
- **Model (`BoardModel.java`):** Lokální reprezentace šachovnice. Slouží primárně pro validaci tahů na straně klienta (např. pro UI highlighting možných tahů) před odesláním na server.
- **View (`GamePanel.java`, `LobbyPanel.java`, `ResultOverlay.java`):** Grafické komponenty Swing. Zajišťují vykreslování šachovnice, interakci s uživatelem a zobrazování výsledků.

Odolnost aplikace

Aplikace implementuje několik mechanismů pro zajištění stability a uživatelského komfortu i v případě nestabilního síťového připojení:

- **Offline Queue:** Klient používá frontu `offlineQueue` pro ukládání zpráv, které se nepodařilo odeslat během výpadku. Po obnovení spojení jsou automaticky odeslány.
- **Automatický Reconnect:** Metoda `attemptReconnect` se pokouší v intervalech obnovit spojení se serverem bez nutnosti zásahu uživatele.
- **Identifikace relace:** Díky unikátnímu `SessionID` server rozpozná vracejícího se hráče a automaticky mu obnoví stav rozehrané partie.

4 Požadavky na překlad a běh

Pro úspěšné sestavení a spuštění obou částí aplikace musí cílové prostředí splňovat následující softwarové nároky:

Server

- **Operační systém:** Linux / Unix (vyžaduje POSIX standard).
- **Kompilátor:** GCC nebo Clang s podporou standardu C11.
- **Knihovny:** Standardní C knihovna + knihovna vláken `pthread`.
- **Nástroje:** GNU Make (pro automatizaci překladu).

Klient

- **Operační systém:** Multiplatformní (Windows, Linux, macOS).
- **Prostředí:** Java Runtime Environment (JRE) verze 8 nebo novější.
- **Nástroje:** Apache Maven (pro správu závislostí a překlad).

5 Postup překladu a spuštění

Následující kroky popisují proces sestavení binárních souborů ze zdrojových kódů a jejich následné spuštění.

5.1 Server

Zdrojové kódy se nachází ve složce `server/src/` a `server/include/`, Makefile soubor se nachází ve složce `server/`.

1. Otevřete terminál a přejděte do složky `server`.
2. Spusťte kompilaci příkazem:

```
make
```

3. Vznikne spustitelný soubor `server.exe`.
4. Spuštění serveru s volitelnými parametry:

```
./server.exe [ip=0.0.0.0] [port=10001] [rooms=10] [players=20]
```

5.2 Klient

Zdrojové kódy se nachází ve složce `client/`.

1. Otevřete terminál a přejděte do složky `client`.
2. Spusťte sestavení pomocí nástroje Maven - Pokud používáte JRE 8, použijte vyhrazený soubor pom_java8.xml:

```
mvn [-s pom_java8.xml] clean package
```

3. Výsledný spustitelný JAR soubor se vytvoří ve složce `target/`.
4. Spuštění klienta:

```
java -jar target/chess-client-1.0.0.jar
```

6 Závěr a zhodnocení

Projekt úspěšně naplňuje zadání semestrální práce. Byla vytvořena stabilní aplikace klient-server umožňující hraní šachů v reálném čase s důrazem na robustnost síťové komunikace.

Během vývoje se podařilo dosáhnout následujících klíčových výsledků:

- **Vlastní protokol:** Byl navržen a implementován textový protokol, který je čitelný pro člověka a zároveň efektivní pro strojové zpracování. Zahrnuje mechanismus potvrzování (ACK) pro zvýšení spolehlivosti nad TCP.
- **Odolnost vůči chybám:** Aplikace zvládá krátkodobé výpadky sítě (automatický reconnect) i pokusy o podvržení nevalidních dat (validace na serveru, odpojení škodlivého klienta).
- **Efektivní paralelizace:** Server využívá vlákna efektivně, což umožňuje obsluhu více herních místností současně bez blokování ostatních uživatelů.
- **Kompletní pravidla:** Je implementována kompletní logika pohybu šachových figur včetně speciálních pravidel (rošáda, en passant).

Aplikace splňuje striktní požadavek na nepoužívání externích síťových knihoven (použity jsou pouze nativní BSD sockety v C a `java.net.Socket` v Javě). Kód je přehledně strukturován do modulů a opatřen podrobnými dokumentačními komentáři.